Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»

Номер зачетной книжки 25350088
Преддипломная практика зачтена с оценкой
_____ (_____)
(цифрой) (прописью)

_____
(подпись руководителя практики от БГУИР)
_____._____.2026

ОТЧЕТ

Минск 2026

СОДЕРЖАНИЕ

Введение 5
1 Обзор источников литературы по теме 7
1.1 Языковые модели и архитектура transformer 7
1.2 Интеллектуальный анализ и обработка документов 9
1.3 Генерация текстового контента с применением языковых моделей 13
2 Выбор и обоснование технических требований 15
2.1 Анализ требований к системе 15
2.2 Обоснование выбора технологического стека 19
3 Проектирование и разработка программного средства 24
3.1 Общая архитектура системы 24
3.2 Проектирование и реализация базы данных 26
3.3 Проектирование и реализация серверной части системы 27
3.4 Проектирование и реализация клиентской части системы 29
3.5 Пазработка модуля семантической сегментации и системы шаблонов оформления 31
3.6 Выводы 32
4 Тестированеи и проверка работоспособности программного средства 33
4.1 Цели, задачи и стратегии тестирования 33
4.2 Результаты тестирования 34
4.3 Выводы 35
5 Руководство по развертыванию и использованию 37
5.1 Развертывание платформы 37
6 Экономическое обоснование разработки и реализации на рынке веб платформы для автоматического форматирования дипломных и курсовых работ 39
6.1 Характеристика программного средства, разрабатываемого для реализации на рынке 39
6.2 Расчет инвестиций в разработку программного средства для его реализации на рынке 41
6.3 Расчет экономического эффекта от реализации программного средства на рынке 43
6.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке 44
Заключение 46
Список литературных источников 47

ВВЕДЕНИЕ

Стремительное развитие технологий обработки естественного языка и появление крупных языковых моделей открывают принципиально новые возможности для автоматизации интеллектуальных задач, в том числе в сфере работы с текстовыми документами. Современные решения на основе трансформерных архитектур демонстрируют высокую точность семантического анализа текста: определения структуры, тематики разделов, стилистических характеристик. Параллельно активно развиваются системы верстки на базе *LaTeX*, позволяющие программно генерировать документы, полностью соответствующие заданным стандартам оформления. Совокупность этих достижений является фундаментальной для построения платформ, способных принять на себя рутинную нагрузку по форматированию академических работ.

Тем не менее подготовка курсовых и дипломных работ в учреждениях образования по-прежнему сопряжена со значительными временными затратами студентов на ручное оформление документов в соответствии с требованиями стандарта предприятия. Базовый цикл включает самостоятельную настройку полей, шрифтов, интервалов, заголовков, нумерации формул, таблиц и рисунков, а также формирование содержания и титульного листа. При этом систематические ошибки форматирования приводят к повторным итерациям проверки нормоконтролёром и увеличивают нагрузку на преподавательский состав. Полнофункциональных программных решений, автоматизирующих весь этот процесс применительно к требованиям СТП БГУИР, на рынке не представлено.

Целью дипломного проектирования является разработка веб-платформы *AutoSTP* − системы для автоматизации оформления курсовых и дипломных работ студентов в соответствии со стандартами БГУИР. Система принимает на вход неоформленный текстовый документ произвольной структуры и формирует на выходе полностью отформатированный документ, готовый к сдаче.

В основу проектирования положены следующие принципы. Первый − сквозная автоматизация: от загрузки исходного файла до экспорта готового документа система выполняет все этапы обработки без обязательного участия пользователя в разметке структуры. Второй − применение *LLM* для семантической сегментации: языковая модель анализирует текст и автоматически определяет границы разделов, что устраняет необходимость предварительного структурирования документа пользователем. Третий − декларативное описание стандарта: правила оформления хранятся в виде конфигурационного шаблона, на основе которого *LaTeX*-механизм генерирует верстку; это позволяет поддерживать произвольные стандарты, не модифицируя программный код. Четвёртый − уровневая модель доступа: базовый тарифный план обеспечивает форматирование по системному шаблону СТП, тогда как премиум-подписка открывает загрузку собственного

стандарта, генерацию персонального шаблона средствами *LLM* и экспорт в *DOCX*.

Пояснительная записка включает следующие разделы. В разделе обзора литературы рассматриваются существующие подходы к автоматизации оформления документов, анализируются частичные аналоги системы − онлайн-редакторы на базе *LaTeX* и текстовые программы общего назначения, а также исследуются современные методы семантического анализа текста с применением языковых моделей. В разделе, посвящённом используемым методам и методикам, описываются принципы семантической сегментации на основе *LLM*, механизм декларативного описания стандарта оформления в виде шаблонов и подход к генерации итогового документа через *LaTeX*. Раздел собственных исследований и проектирования содержит архитектурные решения системы, проектирование базы данных, описание алгоритмов работы ключевых модулей − семантической сегментации, форматирования, интерактивного редактирования и экспорта, − а также описание системы тарифных планов и управления шаблонами. В технико-экономическом обосновании оценивается экономическая эффективность внедрения платформы: сравниваются почасовые затраты на ручное и автоматизированное оформление, обосновывается модель монетизации на основе подписки. Раздел охраны труда посвящён требованиям к условиям работы пользователей при эксплуатации веб-платформы. В заключении подводятся итоги выполненной работы и формулируются направления дальнейшего развития системы.

Данный дипломный проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет XX% (отчет о проверке на заимствования прилагается).

1 ОБЗОР ИСТОЧНИКОВ ЛИТЕРАТУРЫ ПО ТЕМЕ

1.1 Языковые модели и архитектура Transformer

До появления нейросетевых подходов задачи автоматической обработки текста решались преимущественно с помощью систем, основанных на правилах. Такие системы требовали ручного составления словарей, шаблонов и грамматик, что делало их негибкими и трудно масштабируемыми при изменении предметной области. Переломным моментом в развитии обработки естественного языка (*NLP*, от англ. *Natural Language Processing*) стало появление нейросетевых языковых моделей, способных обучаться различным представлениям текста без явного программирования правил.

Первым значимым шагом в этом направлении стала рекуррентная нейронная сеть (*RNN*, от англ. *Recurrent Neural Network*). Новой стала идея того, что *RNN* обрабатывает последовательности данных, сохраняя скрытое состояние, которое передаётся от одного временного шага к следующему. Это позволяет модели учитывать контекст предыдущих токенов при обработке текущего. Однако *RNN* страдает от проблемы затухающего градиента: при обратном распространении ошибки через длинные последовательности градиенты стремительно уменьшаются, что делает обучение на длинных текстах практически невозможным.

Для решения этой проблемы были разработаны сети с долгой краткосрочной памятью (*LSTM*, от англ. *Long Short-Term Memory*) и управляемые рекуррентные блоки (*GRU*, от англ. *Gated Recurrent Unit*). Оба типа сетей вводят специальные механизмы – вентили, – которые управляют потоком информации: определяют, что следует запомнить, что забыть и что передать дальше. *LSTM* и *GRU* значительно улучшили качество работы с длинными последовательностями и на протяжении нескольких лет являлись стандартом в задачах машинного перевода, анализа и классификации текста. Тем не менее их принципиальным ограничением остаётся последовательный характер вычислений: каждый шаг обработки зависит от результата предыдущего, что исключает параллельное обучение и делает обработку длинных документов вычислительно дорогостоящей.

Параллельно развивалась концепция векторных представлений слов – эмбеддингов. Модели *Word2Vec* и *GloVe* позволяли отображать слова в пространство числовых векторов таким образом, что семантически близкие слова оказывались близки и геометрически. Это дало возможность моделям работать со смыслом слов, а не только с их идентификаторами. Однако принципиальный недостаток этих подходов состоит в том, что вектор каждого слова является фиксированным и не зависит от контекста: слово «замок» в значении «крепость» и в значении «запорное устройство» получало одинаковое представление.

Принципиально новый подход к архитектуре нейронных сетей для обработки последовательностей был предложен в 2017 году в статье «*Attention Is All You Need*» [1]. Авторы из *Google Brain* представили архитектуру

Transformer, полностью отказавшуюся от рекуррентности в пользу механизма само-внимания (*self-attention*). Ключевая идея состоит в следующем: при обработке каждого токена в последовательности модель одновременно вычисляет его отношение ко всем остальным токенам этой последовательности, получая взвешенное представление глобального контекста за одну операцию. Это обеспечивает два принципиальных преимущества перед *RNN* и *LSTM*.

Во-первых, вычисления полностью параллельны: поскольку нет зависимости между шагами по времени, все токены обрабатываются одновременно, что кратно ускоряет обучение и инференс. Во-вторых, расстояние между любыми двумя токенами в последовательности с точки зрения информационного пути равно единице вне зависимости от того, насколько далеко они расположены друг от друга в тексте. Это устраняет проблему затухающего градиента и позволяет модели эффективно улавливать дальние зависимости, например связь между местоимением и существительным, разделёнными несколькими предложениями.

Архитектура *Transformer* состоит из энкодера и декодера, каждый из которых построен из одинаковых блоков. Каждый блок включает два основных подслоя: многоголовое само-внимание (*multi-head self-attention*) и позиционно-зависимую полносвязную сеть (*feed-forward network*). Многоголовое внимание означает, что механизм само-внимания применяется параллельно несколько раз с различными обучаемыми проекциями, что позволяет модели одновременно фокусироваться на разных аспектах контекста. После каждого подслоя применяются нормализация и остаточное соединение (*residual connection*), что стабилизирует обучение глубоких сетей. Поскольку *Transformer* не имеет встроенного понятия порядка токенов, в модель вводится позиционное кодирование (*positional encoding*) – вектор, однозначно кодирующий позицию каждого токена в последовательности и суммируемый с его эмбеддингом на входе.

На основе архитектуры *Transformer* сформировалось два ключевых семейства языковых моделей, различающихся используемой частью архитектуры и обучающей задачей.

Первое семейство – модели на основе энкодера, наиболее известным представителем которых является *BERT* (*Bidirectional Encoder Representations from Transformers*), разработанный в *Google* в 2018 году. *BERT* обучается на задаче маскированного предсказания токенов: часть токенов в тексте скрывается, и модель учится восстанавливать их на основе двустороннего контекста. Это делает *BERT* особенно эффективным для задач понимания текста, классификации и извлечения информации. Однако *BERT* не предназначен для генерации текста.

Второе семейство – авторегрессионные модели на основе декодера, такие как *GPT* (*Generative Pre-trained Transformer*). В отличие от *BERT*, *GPT* обучается предсказывать следующий токен по предшествующим, то есть моделирует условное распределение текста. Это делает *GPT* мощным

инструментом для генерации связного текста. Серия *GPT* прошла путь от *GPT-1* (117 миллионов параметров, 2018) до *GPT-4* [2], который содержит предположительно более одного триллиона параметров и демонстрирует способности к следованию сложным инструкциям, выполнению многошаговых рассуждений и работе с длинным контекстом. Ключевым достижением *GPT-4* является возможность использования в режиме инструкционного следования через интерфейс *API*: модель принимает системный промпт с описанием задачи и пользовательский ввод, возвращая структурированный ответ. Именно этот режим применяется в платформе *AutoSTP*.

Сравнение архитектур позволяет обосновать выбор, сделанный при проектировании платформы *AutoSTP*. Рекуррентные сети (*RNN*, *LSTM*, *GRU*) исключаются ввиду принципиальной неспособности к параллельной обработке и ограниченной длины контекста, что критично при работе с документами большого объёма. Энкодерные модели типа *BERT* не подходят для генерации – ни структурированного *JSON*-представления разделов, ни текстовых подсказок по содержанию. Авторегрессионные модели на основе архитектуры *Transformer*, доступные через *API* (в частности, *GPT-4* и его аналоги), удовлетворяют всем требованиям задачи: обрабатывают длинный контекст, генерируют структурированный вывод в формате *JSON* по инструкции, не требуют дообучения для новых задач и предоставляются как готовый сервис с предсказуемым временем отклика. Именно поэтому в подсистемах семантической сегментации и генерации подсказок платформы *AutoSTP* используется *API* языковой модели на основе архитектуры *Transformer*.

1.2 Интеллектуальный анализ и обработка документов

1.2.1 Традиционные подходы к обработке документов. Одним из первых методов автоматической обработки документов стала разработка систем, основанных на правилах (*rule-based systems*), и систем автоматизации с помощью роботизированных процессов (*RPA*, от англ. *Robotic Process Automation*). В таких системах аналитик вручную формулирует набор шаблонов, регулярных выражений и условных правил, которые описывают, где и как искать нужную информацию в документе. Например, правило может гласить: «найти строку, начинающуюся с цифры и точки – это заголовок раздела». Подобный подход работает удовлетворительно для строго стандартизированных документов с предсказуемой структурой. Однако систематический обзор, охвативший 68 исследований за период 2012-2023 годов, установил, что методы, основанные на статических шаблонах и правилах, оказываются несостоятельными при работе с документами, имеющими сложную или нестандартную структуру. Как только документ отклоняется от ожидаемого шаблона – изменяется формулировка заголовка, добавляется нестандартный раздел, нарушается типичный порядок частей –

система даёт сбой и требует ручной доработки правил. Применительно к задаче *AutoSTP* это означало бы следующее: каждый студент имеет собственную манеру структурирования текста, использует разные формулировки заголовков разделов и различный порядок изложения. Построить систему правил, покрывающую всё это многообразие без значительного числа ошибок, практически невозможно [3].

Промежуточным шагом стало применение классических методов машинного обучения –байесовского классификатора, метода опорных векторов – к задачам классификации текстовых фрагментов и извлечения информации. Эти методы требуют ручного конструирования признаков (*feature engineering*): необходимо явно указать модели, на какие характеристики текста обращать внимание – длину предложений, наличие ключевых слов, позицию в документе, использование цифр и т. д. Данный подход превосходит системы правил по гибкости, поскольку модель сама обучается находить характерные паттерны, однако качество полностью определяется набором признаков, а его пополнение требует экспертизы в предметной области.

1.2.2 Переход к нейросетевым методам и интеллект документов. Появление глубокого обучения и, в особенности, архитектуры *Transformer* открыло принципиально новую эпоху в обработке документов. Направление, сформировавшееся на стыке обработки естественного языка, компьютерного зрения и извлечения информации, получило название «интеллект документов» (*document intelligence*). Масштабный обзор, охвативший 322 исследования, опубликованных преимущественно в 2021–2025 годах, зафиксировал следующую трансформацию области: если до 2021 года в задачах понимания документов доминировали специализированные модели на основе *BERT* с ручной разметкой данных, то начиная с 2022–2023 годов на первый план вышли большие языковые модели (*LLM*), демонстрирующие качество обработки документов, недостижимое ранее [4].

В рамках направления «интеллект документов» принято выделять несколько уровней задач. На нижнем уровне находится разбор документа (*document parsing*) – извлечение текстового содержимого с сохранением структуры. На среднем уровне располагаются задачи понимания компоновки (*layout understanding*) – определение типов структурных элементов: заголовков, абзацев, таблиц, подписей к рисункам. На верхнем уровне – семантический анализ: понимание содержательных отношений между разделами, извлечение ключевой информации, генерация ответов на вопросы по документу. Платформа *AutoSTP* работает преимущественно на среднем и верхнем уровнях: ей необходимо не просто извлечь текст из текстового файла, но и определить смысловую роль каждого фрагмента – является ли данный абзац введением, разделом основной части, заключением или библиографической записью.

Обзор систематизирует два принципиальных подхода к организации современных систем обработки документов с применением *LLM*. Конвейерный подход (*pipeline-based*) наследует модульную идею традиционного анализа документов: задача декомпозируется на последовательные этапы – извлечение текста, анализ компоновки, распознавание содержимого, семантическое понимание. Каждый этап обслуживается отдельным специализированным компонентом. Преимуществами этого подхода являются модульная оптимизируемость, интерпретируемость и управляемость. Сквозной подход (*end-to-end*) напротив, передаёт документ непосредственно в *LLM* с соответствующим промптом, получая на выходе структурированный результат за одно обращение. Он менее интерпретируем, однако значительно проще в реализации и нередко превосходит конвейерные системы по качеству на задачах, требующих понимания дальних семантических зависимостей.

1.2.3 Методы сегментации документов. Семантическая сегментация документа – разбиение текста произвольной структуры на логически целостные, осмысленные фрагменты – является одной из ключевых задач в области интеллекта документов и непосредственно реализуется в подсистеме сегментации платформы *AutoSTP*. В исследовательской литературе принято выделять несколько подходов к её решению.

Фиксированная сегментация делит текст на фрагменты заданного размера с возможным перекрытием. Этот подход прост в реализации, однако не учитывает смысловые границы: один фрагмент может обрываться посередине предложения или логического аргумента, что приводит к потере контекста и снижению точности последующего анализа. Для задачи определения границ разделов академической работы данный подход неприменим.

Рекурсивная сегментация последовательно делит документ по иерархии разделителей (сначала по абзацам, затем по предложениям и т. д.) до достижения целевого размера фрагмента. Этот подход лучше сохраняет структуру, однако по-прежнему опирается на формальные признаки, а не на смысловое содержание.

Семантическая сегментация вычисляет эмбеддинги последовательных предложений и определяет границы фрагментов в точках, где косинусное расстояние между соседними предложениями превышает заданный порог, то есть в местах резкого смыслового сдвига.

Наиболее перспективным направлением является гибридная сегментация, объединяющая семантический анализ с учётом пространственной компоновки. Был предложен метод *S2 Chunking*, в котором структурные и семантические отношения между элементами документа кодируются в виде взвешенного графа с применением информации о координатах ограничивающих прямоугольников (*bounding boxes*) и текстовых эмбеддингов. Кластеризация этого графа методом спектральной

кластеризации позволяет получать фрагменты, согласованные одновременно и по смыслу, и по компоновке. Эксперименты, проведённые на датасетах *PubMed* и *arXiv*, показали, что данный подход устойчиво превосходит традиционные методы, особенно для документов со сложной многоколоночной структурой и обилием таблиц и рисунков [5].

Отдельного внимания заслуживает подход с использованием *LLM* непосредственно для сегментации. В данном случае языковой модели передаётся текст документа и системный промпт с инструкцией определить границы логических разделов и вернуть структурированное представление в формате *JSON*. Исследование в рамках вышеупомянутого обзора о применении *LLM* в системах интеллектуальной обработки документов показывает, что современные модели класса *GPT-4* обладают достаточным пониманием структуры академических текстов, чтобы выполнять эту задачу с высокой точностью без специального обучения.

1.2.4 Извлечение информации из документов произвольной структуры. Параллельно с развитием методов сегментации активно исследуется проблема извлечения ключевой информации (*KIE*, от англ. *Key Information Extraction*) из документов произвольной структуры. Систематический обзор методов извлечения информации из неструктурированных документов с использованием ИИ классифицирует подходы к этой задаче по четырём группам.

Первая группа – методы на основе OCR (от англ. Optical Character Recognition) с последующей обработкой правилами. Их достоинство – скорость и предсказуемость на типовых документах. Недостаток – хрупкость при изменении структуры. Вторая группа – нейросетевые методы с ручным конструированием признаков. Более гибкие, но требуют значительных усилий по разметке данных и формированию признаков. Третья группа – методы глубокого обучения на основе *RNN* для совместного анализа текста и компоновки. Существенно превосходят первые две группы, но слабо обобщаются на новые типы документов. Четвёртая, наиболее современная группа – *LLM*-методы, в которых понимание документа достигается путём инструкционного взаимодействия с языковой моделью. Обзор констатирует, что именно статические паттерны и правила оказываются несостоятельными при работе с документами сложной структуры, именно это характеризует входные данные платформы *AutoSTP*, поскольку студенческие работы не имеют единообразной структуры до форматирования.

Важным аспектом является и пространственное понимание документа (*document layout analysis*, *DLA*) – задача определения и классификации структурных элементов: текстовых блоков, заголовков, таблиц, формул. Несмотря на высокие общие способности *LLM* к рассуждению, на задачах *DLA* они уступают специализированным моделям на основе *RoBERTa*-кодировщиков при работе с дефектными или нестандартными документами. Это обстоятельство лежит в основе архитектурного решения *AutoSTP*:

платформа использует *LLM* для семантической сегментации, то есть для определения смысловой роли фрагментов, тогда как физическое извлечение текста из файла осуществляется специализированными библиотеками (*python-docx* например), которые напрямую читают *XML*-структуру документа без потерь.

1.2.5 Применение *LLM* к академическим документам. Академические тексты представляют собой специфический класс документов с характерными чертами: строгой иерархической структурой разделов, наличием формул, таблиц и рисунков с нумерованными подписями, специфическим стилем изложения и библиографическими записями. Обзор о применении *LLM* в системах интеллектуальной обработки документов фиксирует, что модели, прошедшие предобучение на корпусах научных статей (в частности, на базе Semantic Scholar, *arXiv* и *PubMed*), демонстрируют значительно более высокое понимание структуры академических документов по сравнению с моделями общего назначения. Это свойство делает *LLM-API* особенно подходящим инструментом для задачи платформы *AutoSTP* – автоматического определения границ и типов разделов студенческих работ: «введение», «основная часть», «заключение», «список использованных источников» и других предусмотренных СТП элементов.

1.3 Генерация текстового контента с применением языковых моделей

Помимо задач структурного анализа документа, платформа *AutoSTP* включает модуль генерации подсказок по улучшению содержательной части работы. Данный модуль обращается к языковой модели с запросом на анализ текущего содержания раздела и формирование перечня рекомендаций по его улучшению. Теоретическим обоснованием такого подхода служат исследования в области автоматизированного анализа академических текстов с применением *LLM*.

Существует обзор, посвящённый направлению автоматизированного рецензирования научных работ (*ASPR*, от англ. *Automated Scholarly Paper Review*) – автоматического формирования содержательной обратной связи на основе *LLM*. Авторы отслеживают эволюцию этого направления: если ранние системы опирались на поверхностный анализ лексики и простые эвристики, то начиная с 2023 года ключевым инструментом стало инструкционное взаимодействие с крупными языковыми моделями. Обзор фиксирует, что современные модели, в первую очередь *GPT-4* и его аналоги, способны генерировать рецензии, включающие оценку полноты изложения, выявление логических пробелов, предложения по расширению теоретической базы, и по ряду показателей эти рецензии сопоставимы с рецензиями людей-экспертов начального уровня [6].

Принципиально важным для архитектуры модуля подсказок *AutoSTP* является выявленная в обзоре закономерность: качество генерируемой обратной связи критически зависит не от размера модели, а от качества промпта. Промпт – это текстовая инструкция, которая передаётся языковой модели вместе с обрабатываемым контентом и определяет, какой именно результат ожидается на выходе. Направление разработки эффективных промптов получило название *prompt engineering* (проектирование подсказок).

В контексте генерации академического контента выделяют несколько ключевых техник *prompt engineering*. Нулевой промпт (*zero-shot prompting*) означает выполнение задачи только по словесному описанию, без каких-либо примеров. Благодаря масштабному предобучению на корпусах научных текстов современные *LLM* справляются с задачами анализа академических разделов в режиме *zero-shot* без дополнительного обучения. Промпт с примерами (*few-shot prompting*) дополнительно включает в запрос несколько образцов желаемого вывода, что позволяет точнее задать стиль и формат ответа. Цепочечное мышление (*chain-of-thought prompting*) побуждает модель явно излагать промежуточные рассуждения перед формулировкой ответа, что повышает качество на задачах, требующих многошагового анализа.

Немаловажным аспектом применения *LLM* для генерации академического контента является проблема галлюцинаций – генерации правдоподобно звучащей, но фактически недостоверной информации. Обзор особо подчёркивает эту проблему применительно к *ASPR*: модель может формулировать конкретные рекомендации, ссылаясь на несуществующие источники или неверные факты. Именно поэтому в *AutoSTP* принято архитектурное решение генерировать не готовый текст для вставки в работу, а лишь подсказки – перечень направлений для развития раздела, окончательное содержание которых студент формулирует самостоятельно. Это снижает риск внесения недостоверной информации в академическую работу и одновременно соответствует педагогической логике: система помогает студенту думать, а не думает вместо него.

2 ВЫБОР И ОБОСНОВАНИЕ ТЕХНИЧЕСКИХ ТРЕБОВАНИЙ

2.1 Анализ требований к системе

2.1.1 Функциональные требования определяют полный перечень действий, которые система обязана выполнять для достижения целей, сформулированных в техническом задании. На основании анализа предметной области, изучения процесса оформления курсовых и дипломных работ студентами БГУИР, а также требований СТП 01-2024 выделены следующие группы функциональных требований, организованные по подсистемам платформы AutoSTP.

Управление пользователями и аутентификация. Система должна обеспечивать регистрацию нового пользователя по адресу электронной почты и паролю с проверкой уникальности адреса в базе данных. При регистрации пароль подвергается хешированию с добавлением случайной соли перед сохранением, что исключает хранение паролей в открытом виде. После успешной аутентификации система выдаёт пару токенов: токен доступа со временем жизни один час и токен обновления со временем жизни тридцать дней. Все запросы к защищённым ресурсам сопровождаются токеном доступа в *HTTP*-заголовке *Authorization*, который проверяется серверным *middleware* до передачи запроса обработчику. Система реализует разграничение прав доступа: пользователь с ролью «Пользователь» имеет доступ исключительно к своим документам и шаблонам, пользователь с ролью «Администратор» имеет доступ к шаблонам и документам всех пользователей.

Управление документами. Система должна принимать загружаемые пользователем текстовые документы в форматах *DOCX* и *TXT* через *HTTP*-запрос типа *multipart/form-data*. При загрузке сервер извлекает текстовое содержимое документа, сохраняет исходный файл в файловом хранилище и создаёт запись документа в базе данных, включающую идентификатор владельца, название, дату создания, дату последнего изменения и структурированное содержимое в формате *JSON*. Система должна реализовывать автоматическое сохранение текущего состояния редактируемого документа с интервалом не более пяти минут для предотвращения потери данных при сбоях. Пользователю должна быть доступна история ранее загруженных и обработанных документов с возможностью повторного просмотра и скачивания. Каждый пользователь имеет доступ исключительно к своим документам; доступ к документам других пользователей исключён на уровне серверной авторизации.

Семантическая сегментация документа. Система должна выполнять автоматическое определение логической структуры загруженного документа произвольной формы без предварительной ручной разметки пользователем. Подсистема формирует структурированный промпт для языковой модели, включающий перечень стандартных разделов курсовой и дипломной работы

согласно СТП, инструкцию по формату вывода и исходный текст документа, после чего отправляет запрос к *API* языковой модели. Результатом сегментации является *JSON*-объект, содержащий список разделов с указанием типа каждого (введение, основная часть, заключение, список использованных источников и прочие элементы, предусмотренные СТП), а также начальной и конечной позиции каждого раздела в исходном тексте. Полученный *JSON* проходит валидацию на соответствие ожидаемой схеме; при обнаружении несоответствий выполняется повторный запрос с уточнёнными инструкциями, но не более двух раз. Результат сегментации сохраняется в базе данных и передаётся в подсистему форматирования.

Форматирование документа по шаблону СТП. Система должна применять к структурированному содержимому документа правила оформления, закодированные в конфигурационном шаблоне СТП в виде *JSON*-объекта формата «поле - значение». Шаблон содержит следующие параметры: размеры полей страницы, тип и кегль основного шрифта, межстрочный интервал, абзацный отступ, правила оформления заголовков разделов и подразделов, формат нумерации страниц, правила оформления таблиц, рисунков и формул, требования к библиографическим записям и так далее.

Интерактивное редактирование документа. Система должна предоставлять пользователю визуальный редактор с постраничным отображением документа в виде, приближённом к финальному результату после экспорта. Редактор обеспечивает следующую функциональность:

– редактирование текстового содержимого непосредственно в визуальном представлении с сохранением форматирования;

– вставку изображений;

– добавление таблиц с возможностью задания числа строк и столбцов, редактирования содержимого ячеек;

– редактирование библиографических записей через специализированную форму с полями для всех обязательных элементов описания источника;

– управление структурой документа: изменение порядка разделов, добавление новых разделов, удаление существующих.

Отображение всех изменений в интерфейсе должно происходить без полной перезагрузки страницы.

Генерация подсказок по дополнению содержания работы. По запросу пользователя система должна анализировать текущее содержимое выбранного раздела документа и формировать перечень рекомендаций по его развитию с использованием языковой модели. Промпт, передаваемый языковой модели, включает название раздела, общую тему работы и текущий текст раздела. Результат представляет структурированный перечень направлений для развития содержания: каких аспектов не хватает, какие подтемы следует раскрыть, какие связи с другими разделами необходимо установить.

Подсказки отображаются в боковой панели редактора. Генерация подсказок для одного раздела должна выполняться за время не более пяти секунд.

Экспорт готового документа. Система должна формировать финальную версию отформатированного документа в одном из двух форматов по выбору пользователя: *PDF* или *DOCX*. Экспортный файл должен полностью соответствовать требованиям СТП, корректно отображать кириллические символы, математические формулы и специальные символы, а также включать все графические элементы. Сформированный файл предоставляется пользователю для скачивания через веб-интерфейс. Время формирования файла объёмом до сорока страниц не должно превышать одной минуты, до ста страниц – двух минут.

Управление шаблонами оформления. Система должна поддерживать два типа конфигурационных шаблонов СТП. Системный шаблон создаётся и редактируется администратором платформы и доступен всем пользователям по умолчанию; он кодирует требования СТП 01-2024 БГУИР. Персональный шаблон формируется по инициативе пользователя: пользователь загружает документ стандарта оформления в формате *DOCX*, языковая модель автоматически извлекает из него все формализуемые правила и создаёт шаблон в формате «поле - значение». Персональный шаблон доступен исключительно пользователю, создавшему его. После автоматического формирования пользователь может вручную скорректировать любые поля шаблона через специализированный табличный интерфейс. При запуске форматирования пользователь выбирает, какой шаблон применять – системный или персональный.

2.1.2 Нефункциональные требования определяют качественные характеристики системы: её производительность, надёжность, безопасность, масштабируемость и точность обработки данных.

Требования к производительности. Платформа обрабатывает запросы разной вычислительной стоимости, поэтому устанавливаются раздельные временные регламенты для каждой категории операций. Для лёгких операций (аутентификация, чтение метаданных, сохранение изменений в редакторе) время отклика *API* не должно превышать пятисот миллисекунд. Для тяжёлых операций, включающих обращение к языковой модели, устанавливаются следующие предельные значения:

– семантическая сегментация документа объёмом до ста страниц – не более трёх минут;

– генерация подсказок для одного раздела – не более пяти секунд;

– экспорт документа объёмом до сорока страниц – не более одной минуты, до ста страниц – не более двух минут;

– обработка документа объёмом до ста страниц модулем форматирования – не более десяти минут.

Система должна поддерживать одновременную работу не менее десяти пользователей без деградации производительности ниже установленных

пороговых значений. Максимальный допустимый размер загружаемого файла составляет тридцать мегабайт.

Требования к надёжности. Система должна гарантировать сохранность данных пользователей при возникновении непредвиденных сбоев. Для этого реализуются:

1 Автоматическое сохранение текущего состояния редактируемого документа с интервалом не более пяти минут; при восстановлении работы после сбоя пользователю предоставляется возможность продолжить работу с последней автоматически сохранённой версией;

2 Автоматическое резервное копирование базы данных с ежедневной периодичностью; резервные копии хранятся в течение тридцати дней;

3 Режим работы системы – круглосуточный, семь дней в неделю.

Требования к безопасности. Вся передача данных между клиентом и сервером должна осуществляться исключительно по протоколу *HTTPS*. Пароли пользователей хранятся исключительно в хешированном виде с использованием криптографически стойкого алгоритма. Система ведёт журнал аудита, фиксирующий все попытки входа (успешные и неуспешные), а также время создания, изменения и удаления документов, журнал хранится в течение месяца. Сессия пользователя автоматически завершается после одного часа неактивности. Требования к паролям: минимальная его длина составляет восемь символов, обязательное наличие строчных и прописных букв, а также цифр.

Требования к точности обработки. Точность определения структуры документа модулем семантической сегментации (доля разделов, тип которых определён верно, от общего числа разделов в документе) должен быть не менее восьмидесяти процентов. Доля документов, полностью соответствующих требованиям СТП после автоматического форматирования, – не менее девяноста пяти процентов.

Требования к масштабируемости. Архитектура системы должна допускать горизонтальное масштабирование серверной части при росте нагрузки без архитектурных изменений. Это обеспечивается за счёт отсутствия сессионного состояния на сервере и контейнеризации компонентов посредством *Docker*, что позволяет запускать несколько экземпляров серверного приложения за балансировщиком нагрузки.

2.1.3 Требования к пользовательскому интерфейсу. Пользовательский интерфейс платформы должен соответствовать принципам юзабилити, обеспечивая интуитивно понятную работу для студентов, не обладающих специальными техническими знаниями. Ниже перечислены требования к интерфейсу.

Доступность и кроссплатформенность. Система должна быть доступна через стандартный веб-браузер без установки дополнительного программного обеспечения. Интерфейс должен корректно отображаться как на настольных компьютерах, так и на ноутбуках.

Редактор документа. Редактор должен отображать документ постранично в виде, приближённом к финальному результату после экспорта. Пользователю должна быть доступна настройка масштаба отображения документа в диапазоне от десяти до двухсот процентов. Основная область редактирования располагается в центральной части экрана; инструментальные панели размещаются сверху и/или сбоку от области редактирования. Все интерактивные элементы интерфейса должны иметь визуальную обратную связь при наведении указателя мыши и при нажатии.

Стилистика и эстетика. Визуальный стиль интерфейса должен быть выдержан в единой цветовой гамме с использованием основных цветов: белого, серого и синего. Дизайн должен придерживаться минималистичного стиля без избыточных декоративных элементов. Количество одновременно отображаемых элементов управления должно быть минимизировано для снижения когнитивной нагрузки на пользователя. Использование иконографии для повышения информативности элементов интерфейса является обязательным.

Обратная связь и информирование пользователя. При выполнении длительных операций (семантической сегментации, генерации документа, экспорта) – интерфейс должен отображать индикатор выполнения или текстовое сообщение о статусе процесса. При возникновении ошибок пользователю должно выводиться информативное сообщение, описывающее причину ошибки в понятных терминах, без технических деталей реализации. Успешное автосохранение документа должно подтверждаться ненавязчивым визуальным уведомлением.

2.2 Обоснование выбора технологического стека

2.2.1 Для реализации серверной части платформы рассматривались три *Python*-фреймворка: *Django*, *Flask* и *FastAPI*. *Django* предоставляет наиболее полный набор встроенных инструментов и хорошо подходит для классических веб-приложений с серверным рендерингом, однако его синхронная природа создают затруднения при работе с длительными асинхронными операциями, к которым относятся запросы к *LLM API*. *Flask* является минималистичным микрофреймворком и требует самостоятельной сборки из сторонних библиотек для большинства задач, что увеличивает трудоёмкость разработки без очевидных преимуществ.

Для платформы *AutoSTP* выбран фреймворк *FastAPI*. Ключевым аргументом является нативная поддержка асинхронного выполнения через механизм *async/await* в сочетании с *ASGI*-сервером *Uvicorn*: во время ожидания ответа от *LLM API*, занимающего от двух до тридцати секунд, серверный процесс не блокируется и продолжает обрабатывать входящие запросы других пользователей. Дополнительными преимуществами являются автоматическая генерация *OpenAPI*-документации на основе аннотаций типов и встроенная валидация входных данных через *Pydantic*.

2.2.2 Для клиентской части рассматривались два фреймворка: *React* и *Vue.js*. Оба обеспечивают быстрое обновление интерфейса без перезагрузки страницы и пригодны для построения одностраничных приложений. Выбран *React*. Экосистема *React* включает специализированные библиотеки для построения текстовых редакторов (*Slate.js* и *Draft.js*), поддерживающие произвольные схемы документа, вложенные узлы, кастомные форматы и рендеринг специальных элементов (таблицы, формулы, рисунки). Для предпросмотра *PDF*-файлов в браузере доступна библиотека *PDF.js*. Размер сообщества и объём документации у *React* превосходят *Vue.js*, что важно при разработке нетривиальных *UI*-компонентов редактора.

2.2.3 Для хранения данных рассматривались три СУБД: *PostgreSQL*, *MySQL* и *MongoDB*. *MySQL* является зрелым решением, однако уступает *PostgreSQL* по функциональности: в частности, не имеет полноценного типа *JSONB* с индексированием, что критично для хранения структурированного содержимого документов. *MongoDB* – документоориентированная СУБД, хорошо подходящая для хранения вложенных *JSON*-структур, однако менее развитая экосистема *ORM*-библиотек для *Python* делает её менее предпочтительной в данном контексте.

Выбрана СУБД *PostgreSQL*. Тип *JSONB* позволяет эффективно хранить структурированное содержимое документа и шаблоны оформления в виде *JSON*-объектов с поддержкой индексирования и операторов запросов внутри *JSON*-полей. Полная поддержка транзакций *ACID* гарантирует целостность данных при одновременных операциях автосохранения и форматирования. *PostgreSQL* имеет зрелую асинхронную интеграцию с *Python* через библиотеки *asyncpg* и *SQLAlchemy*, что согласуется с асинхронной архитектурой серверной части.

2.2.4 Для задач семантической сегментации и генерации подсказок рассматривались два варианта: развёртывание языковой модели локально на сервере платформы и использование модели через внешний *API*.

Локальное развёртывание открытых моделей (семейства *LLaMA*, *Mistral* и аналогичных) обеспечивает независимость от сторонних сервисов и отсутствие переменных затрат на *API*-запросы. Однако для достаточного качества семантического анализа академических текстов на русском языке требуются модели с числом параметров от двадцати семи миллиардов и выше, что предъявляет к серверу обязательное требование наличия *GPU* с объёмом видеопамяти не менее шестнадцати гигабайт. Это существенно увеличивает стоимость инфраструктуры и выходит за рамки требований технического задания.

Выбрана интеграция через внешний *API* языковой модели (в частности, *GPT-4o* или аналог с сопоставимыми характеристиками). Данный подход не требует *GPU* на стороне платформы: вся вычислительная нагрузка по обработке текста ложится на инфраструктуру провайдера. Модели класса

GPT-4 демонстрируют достаточную точность семантического анализа русскоязычных академических текстов без дообучения и поддерживают структурированный *JSON*-вывод по инструкции. Взаимодействие с *API* реализуется асинхронно, что согласуется с архитектурой серверной части на основе *FastAPI*.

2.3 Обоснование модели тарификации и управления доступом

Для обеспечения надежного функционирования веб-платформы *AutoSTP*, защиты пользовательских данных и достижения экономической рентабельности проекта была спроектирована комплексная модель управления доступом и тарификации. Архитектурные и экономические решения в рамках данной модели базируются на специфике целевой аудитории и характере использования предоставляемых вычислительных мощностей.

Управление доступом и безопасность данных. Архитектура безопасности платформы строится на принципе строгой изоляции пользовательских данных и ролевой модели доступа. В системе выделены два основных уровня привилегий:

1 Уровень администратора: обладает эксклюзивными правами на создание, редактирование и глобальное обновление системного конфигурационного шаблона. Данный шаблон строго кодирует требования актуального стандарта СТП 01-2024 БГУИР и доступен всем пользователям по умолчанию для форматирования базовых работ.

2 Уровень пользователя: обеспечивает доступ к загрузке документов, интерактивному редактору, семантической сегментации и экспорту.

Критически важным аспектом управления доступом является обеспечение конфиденциальности академических работ. Базовая архитектура платформы гарантирует, что каждый пользователь имеет доступ исключительно к своим документам. Доступ к файлам других пользователей полностью исключен на уровне серверной авторизации. Для защиты учетных записей вся передача данных между клиентом и сервером осуществляется по защищенному протоколу *HTTPS*, пароли пользователей хранятся исключительно в хешированном виде, а система ведет журнал аудита сессий.

Обоснование модели тарификации. Специфика эксплуатации системы *AutoSTP* диктует необходимость гибкого подхода к монетизации. Целевая аудитория (студенты 3–4 курсов и магистранты) обращается к платформе не на постоянной основе, а эпизодически. Пик востребованности вычислительных мощностей и *API* языковых моделей приходится на предсессионные периоды, два раза в год.

Внедрение классической модели ежемесячной подписки в данных условиях экономически нецелесообразно, поскольку неизбежно приведет к массовому оттоку пользователей в межсессионный период и снижению лояльности аудитории. В связи с этим в качестве основной выбрана

транзакционная модель оплаты (*pay-per-use*), которая максимально точно отражает характер потребления услуг. Более подробно экономическое обоснование расписано в главе 6.

2.4 Требования к инфраструктуре и развертыванию

Для обеспечения отказоустойчивости, масштабируемости и высокой производительности веб-платформы спроектирована архитектура инфраструктуры. Выбор аппаратных и программных средств базируется на результатах профилирования нагрузки, возникающей при семантическом анализе текста, взаимодействии с *API* языковых моделей и программной генерации документов.

2.4.1 Требования к серверной инфраструктуре. Поскольку платформа переносит всю вычислительную нагрузку по обработке академических текстов на сторону сервера, к аппаратному обеспечению (*hardware*) предъявляются строгие минимальные требования:

1 Центральный процессор: не менее 4 ядер. Многоядерная архитектура критически важна для асинхронной обработки запросов сервером. Разделение потоков позволяет параллельно обслуживать интерактивный редактор множества пользователей и выполнять ресурсоемкие фоновые задачи, не блокируя основной цикл событий.

2 Оперативная память: не менее 16 ГБ. Значительный объем оперативной памяти обусловлен необходимостью загрузки в память и парсинга объемных неструктурированных документов (до 100 страниц для дипломных проектов). Кроме того, *RAM* резервируется под кэширование промежуточных результатов обработки и работу изолированных контейнеров.

3 Графический ускоритель (*GPU*): опционально. Основная нагрузка по генерации текста делегируется внешнему *API* языковой модели.

2.4.2 Технологии виртуализации и контейнеризации. В качестве базовой технологии развертывания приложения выбран *Docker*.

Использование контейнеризации позволяет изолировать окружение: *Backend*, Frontend и система управления базами данных изолированы друг от друга в собственных контейнерах. Это исключает конфликты версий библиотек и зависимостей. Также конфигурация инфраструктуры описывается декларативно (в виде файлов *Dockerfile* и *docker-compose.yml*), что гарантирует идентичность поведения системы на этапах разработки, тестирования и промышленной эксплуатации.

Так как платформа требует наличия специфического программного обеспечения для верстки, контейнеризация также позволяет инкапсулировать эти тяжеловесные зависимости внутрь образа, избегая «засорения» сервера и упрощая миграцию проекта на другие узлы.

Роль единой точки входа для всех клиентских запросов выполняет веб-сервер *Nginx*, сконфигурированный в качестве прокси-сервера. Его применение технически обосновано следующими факторами:

1 Маршрутизация и отдача статики: *Nginx* эффективно маршрутизирует *API*-запросы к *backend*-контейнерам, а статические ресурсы *frontend*-приложения отдает напрямую из файловой системы хоста или кэша, что существенно снижает время загрузки интерфейса.

2 Балансировка и защита: сервер обеспечивает защиту от базовых *DDoS*-атак посредством ограничения количества запросов с одного *IP*-адреса в единицу времени, что особенно актуально в периоды пиковой сессионной нагрузки.

Выбранный стек технологий и архитектурные решения гарантируют, что система способна стабильно обслуживать целевую аудиторию студентов даже в периоды максимальной академической нагрузки, сохраняя при этом потенциал для горизонтального и вертикального масштабирования.

3 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА

3.1 Общая архитектура системы

При проектировании веб-платформы AutoSTP была выбрана клиент-серверная архитектура, которая наиболее полно отвечает функциональным и нефункциональным требованиям, сформулированным в разделе 2 настоящей пояснительной записки. Данное архитектурное решение обеспечивает четкое разделение ответственности между компонентами, высокую масштабируемость, удобство сопровождения и возможность независимой разработки клиентской и серверной частей системы.

Архитектура платформы относится к типу Single Page Application (SPA) и построена по классической трёхуровневой модели с дополнительным выделением слоя внешних интеллектуальных сервисов.

3.1.1 Выбор архитектурного стиля. Анализ возможных вариантов построения системы (монолитная архитектура, микросервисная и клиент-серверная) показал, что клиент-серверный подход является оптимальным на текущем этапе развития проекта. Монолитная архитектура была отвергнута вследствие высокой степени связности компонентов и ожидаемых сложностей при дальнейшем масштабировании и добавлении новой функциональности. Полноценная микросервисная архитектура на данном этапе признана избыточной, поскольку количество доменных сущностей относительно невелико, а требования к независимому масштабированию отдельных модулей пока не являются критическими.

Клиент-серверная архитектура с четко выделенным RESTful API позволила добиться значительных преимуществ. Она обеспечивает независимую разработку и тестирование frontend- и backend-частей, упрощает повторное использование серверного API в будущем и создает хорошие предпосылки для горизонтального масштабирования системы в периоды пиковой нагрузки.

3.1.2 Высокоуровневая структура архитектуры. Система состоит из нескольких основных слоев. Клиентский слой отвечает за представление информации и непосредственное взаимодействие с пользователем. Серверный слой реализует всю бизнес-логику, включая обработку документов, семантический анализ, применение правил форматирования и обеспечение безопасности. Слой данных обеспечивает надежное хранение информации в реляционной базе данных. Отдельный слой составляют внешние сервисы, среди которых важнейшим является сервис доступа к языковым моделям через OpenRouter API.

3.1.3 Организация взаимодействия компонентов. Взаимодействие между клиентской и серверной частями осуществляется исключительно по протоколу HTTPS через RESTful API. Аутентификация и авторизация пользователей построены на основе JWT-токенов с разделением на access-токены ограниченного срока действия и долгосрочные refresh-токены. Все операции, связанные с документами конкретного пользователя, защищены строгой проверкой прав доступа на уровне сервера.

Особое внимание при проектировании уделено обработке длительных операций. Семантическая сегментация документов, генерация рекомендаций по содержанию и процессы финального форматирования и экспорта выполняются в асинхронном режиме. Это позволяет сохранять высокую отзывчивость пользовательского интерфейса даже при работе с объемными дипломными проектами.

3.1.4 Обеспечение масштабируемости, отказоустойчивости и безопасности. Для достижения требуемого уровня масштабируемости и отказоустойчивости все компоненты платформы контейнеризированы с использованием Docker. В качестве фронтенд-сервера и reverse-прокси используется Nginx, который обеспечивает балансировку нагрузки, эффективную отдачу статических ресурсов и поддержку HTTPS.

Backend-часть полностью реализована на асинхронной модели выполнения, что особенно важно при взаимодействии с внешними языковыми моделями, время ответа которых может существенно варьироваться. Отсутствие состояния на сервере (stateless architecture) позволяет легко масштабировать количество экземпляров backend-приложения.

Вопросам информационной безопасности уделено приоритетное внимание. Все передаваемые данные защищаются протоколом HTTPS, пароли хранятся исключительно в хешированном виде, реализована ротация и отзыв токенов, а также ведется детальное журналирование значимых действий пользователей.

Общая архитектура платформы AutoSTP в полной мере соответствует принципам, заявленным в постановке задачи дипломного проекта: обеспечению сквозной автоматизации процесса оформления работ, использованию декларативных шаблонов для описания стандартов СТП БГУИР, эффективному применению современных языковых моделей и созданию удобного интерфейса для студентов, не обладающих углубленной технической подготовкой.

В следующих подразделах текущего раздела будут подробно рассмотрены проектирование базы данных, архитектура и реализация backend- и frontend-компонентов, алгоритмы ключевых модулей системы, система шаблонов оформления, интерактивный редактор документов и модуль экспорта.

3.2 Проектирование и реализация базы данных

Проектирование базы данных являлось одним из ключевых этапов разработки платформы, поскольку корректная структура хранения данных в том числе определяет эффективность работы всех основных модулей системы: управления документами, семантической сегментации, форматирования, версионирования и обеспечения безопасности. Созданная база данных обеспечивает надежное хранение как структурированных метаданных, так и сложных иерархических структур документов, поддерживает высокую скорость чтения и записи при одновременной работе множества пользователей, а также успешно работает с JSON-подобными данными.

3.2.1 Общая структура базы данных. Схема базы данных разрабатывалась с учетом принципа нормализации, где это требовалось для производительности. Было выделено несколько основных групп сущностей:

– сущности управления доступом и безопасностью (роли пользователей, пользователи, активные сессии),

– сущности управления документами и их версиями,

– сущности внутренней структуры документа (разделы, текстовые элементы, таблицы, изображения, библиографические ссылки),

– сущности системы шаблонов оформления,

– сущности аудита.

Все таблицы спроектированы с использованием уникальный идентификаторов в качестве первичных ключей. Для обеспечения каскадного удаления при удалении документа или пользователя применяются соответствующие *FOREIGN KEY* с *ON DELETE CASCADE*.

3.2.2 Основные таблицы и их взаимосвязи. Центральное место в схеме занимает таблица *documents*, которая хранит основную информацию о каждом документе пользователя. Каждая запись в этой таблице связана с пользователем через внешний ключ. Для поддержки версионирования документов введена отдельная таблица *document_versions*, содержащая снимки состояния документа в определенный момент времени. Связь между этими сущностями реализована через поле внешнего ключа, что позволяет эффективно получать актуальную версию документа.

Структура самого документа представлена в нормализованном виде. Таблица *sections* описывает иерархию разделов работы с поддержкой вложенности через поле *parent_id*. Каждый раздел может содержать множество текстовых элементов (*text_elements*), таблиц (*document_tables*), изображений (*document_images*), формул (*document_formulas*). Для хранения содержимого таблиц используется таблица *table_cells*. Библиографические ссылки и цитаты в тексте вынесены в отдельные таблицы *bibliographic_references* и *citations* для обеспечения удобства управления списком литературы .

Отдельной сущностью являются шаблоны оформления (*templates*). Они хранятся в *JSONB*-формате и могут быть как системными (доступными всем пользователям), так и персональными (принадлежащими конкретному пользователю). Это позволило реализовать принцип декларативного описания стандартов СТП.

Для обеспечения безопасности и контроля действий пользователей предусмотрены таблицы *roles*, *sessions* и *audit_logs*. Таблица *sessions* используется для хранения и отзыва *refresh*-токенов, а *audit_logs* фиксирует все значимые операции с документами и шаблонами, действиями пользователей.

3.2.3 Реализация моделей данных. Все сущности базы данных реализованы в виде *ORM*-моделей в асинхронном режиме. Модели разделены по логическим модулям, представленным в виде *python* скриптов., что обеспечивает удобство сопровождения и позволяет легко расширять схему при добавлении новой функциональности.

Особое внимание было уделено корректной обработке циклической связи между таблицами документов и их версионированием. При создании нового документа сначала создается запись в *documents* без указания текущей версии, затем формируется первая версия в *document_versions*, после чего обновляется поле *current_version_id*.

Для работы с *JSONB*-полями (*metadata*, *snapshot*, *formatting*, *template_json* и другими) активно использовались возможности *PostgreSQL*, включая создание *GIN*-индексов для ускорения поиска.

3.2.4 Инициализация и тестирование базы данных. Инициализация схемы базы данных осуществляется автоматически при запуске приложения.

Для обеспечения качества реализации была разработана и успешно выполнена комплексная серия *unit*-тестов, проверяющих корректность создания моделей, работу связей, каскадное удаление, а также работу с иерархической структурой разделов и содержимым таблиц. Тестирование проводилось на отдельной тестовой базе данных, что исключило влияние на основную среду разработки.

3.3 Проектирование и реализация серверной части системы

Разработка серверной части платформы проводилась поэтапно. Основной целью данного этапа являлось создание надёжного, масштабируемого и безопасного серверного приложения, способного обрабатывать запросы пользователей, обеспечивать взаимодействие с языковыми моделями, управлять документами и шаблонами, а также гарантировать целостность и конфиденциальность данных.

3.3.1 Разработка бэкенда началась с инициализации структуры проекта. Была создана директория *backend/*, внутри которой сформирована стандартная структура приложения *API*. Были созданы директории для эндпоинтов, для конфигурации, безопасности и настроек, для *ORM*-моделей, для *Pydantic*-схем запросов и ответов, отдельная директория для бизнес-логики, и еще одна для вспомогательных утилит.

Далее была настроена конфигурация приложения через класс *Settings* на базе *Pydantic*, поддерживающая загрузку параметров из файла с переменными окружения. Особое внимание уделялось подключению к базе данных с использованием асинхронного движка. При запуске приложения в режиме разработки автоматически создавались все таблицы базы данных.

Параллельно был подготовлен *Docker*-контейнер настроен файл *docker-compose.yml*, позволяющий запускать одновременно базу данных и серверное приложение с поддержкой *hot-reload*.

3.3.2 Следующим логическим шагом стала разработка модуля безопасности. В первую очередь был реализован функционал регистрации и аутентификации пользователей. Создан эндпоинт *POST /api/v1/auth/register*, выполняющий валидацию пароля (минимум 8 символов, наличие строчных, прописных букв латиницы и цифр), хеширование пароля и сохранение пользователя в базе данных с ролью *student* по умолчанию.

Затем был разработан эндпоинт *POST /api/v1/auth/login*, возвращающий пару токенов: короткоживущий *access*-токен (срок действия 1 час) и долгоживущий *refresh*-токен (30 дней). *Refresh*-токены сохраняются в отдельной таблице *sessions* для возможности их отзыва.

Была также реализована защита маршрутов. Все защищённые эндпоинты требуют *Bearer* заголовка *Authorization*. Реализован механизм обновления токенов и выхода из системы, отзывающий текущую сессию.

3.3.3 После обеспечения безопасности началась работа с основным доменом приложения, документами пользователей. Первым был реализован эндпоинт загрузки документов. Он поддерживает форматы *DOCX* и *TXT*. При загрузке *TXT*-файла происходит автоматическое преобразование его в минимальный *DOCX*-документ с сохранением структуры абзацев. Оригинальный файл всегда сохраняется на диске в формате *DOCX*, а извлечённый текст и метаданные записываются в базу данных.

Далее были разработаны эндпоинты получения списка документов пользователя с поддержкой пагинации и фильтрации по названию, получения информации об отдельном документе, а также его удаления. Все операции защищены проверкой прав владения: пользователь может работать только со своими документами.

Для поддержки интерактивного редактирования был реализован *PUT* эндпоинт, позволяющий сохранять актуальный *JSON-snapshot* состояния

документа. При каждом обновлении создаётся новая версия документа в таблице версий, что обеспечивает возможность отката изменений.

3.3.4 Интеграция с языковыми моделями и система шаблонов. Важным этапом стало подключение внешнего сервиса *OpenRouter* для работы с языковыми моделями. В отдельном модуле был создан асинхронный клиент с поддержкой *retry*-механизма (экспоненциальная задержка), таймаутами и конфигурацией модели через переменные окружения. Были подготовлены базовые шаблоны промптов также в отдельной директории.

Параллельно реализована система шаблонов оформления. Разработана базовая *JSON*-схема шаблона, включающая как правила форматирования (поля страницы, шрифты, интервалы и т.д.), так и блок структуры, описывающий эталонный состав разделов работы согласно СТП БГУИР. Пользователи могут создавать персональные шаблоны, в том числе с помощью генерации через *LLM* на основе загруженного примера оформления.

3.4 Проектирование и реализация клиентской части системы

Клиентская часть веб-платформы была спроектирована и реализована как одностраничное приложение, что позволило обеспечить высокую отзывчивость интерфейса, плавную навигацию и удобство работы с интерактивным редактором документов.

3.4.1 Архитектура клиентского приложения. Клиентская часть построена по компонентному подходу с чётким разделением ответственности. Все компоненты так же разделены на несколько основных групп: авторизации и регистрации, управления документами (список, создание, удаление), компоненты основного редактора документов, работы с шаблонами оформления, а также общие UI-компоненты (уведомления, индикаторы загрузки, модальные окна).

Также реализована маршрутизация, что позволило создать удобную навигацию между страницами без перезагрузки приложения. Все запросы к серверу защищены автоматическим добавлением *JWT*-токена в заголовок *Authorization*. При получении ошибки 401 происходит автоматический выход пользователя из системы.

3.4.2 Разработка *frontend* части началась с создания страниц регистрации и входа. Были реализованы формы с клиентской валидацией пароля в соответствии с требованиями безопасности (минимум 8 символов, наличие разных регистров и цифр). После успешной авторизации *access*-токен и *refresh*-токен сохраняются в *localStorage*, а информация о пользователе в контекст клиента.

Был создан защищённый маршрутный компонент, который проверяет наличие валидного токена и перенаправляет неавторизованных пользователей

на страницу входа. Реализован механизм автоматического обновления *access*-токена при его истечении с использованием *refresh*-токена.

3.4.3 Следующим этапом стала реализация главной страницы и списка документов пользователя. Были созданы компоненты для отображения списка работ с поддержкой пагинации и поиска по названию. Добавлена возможность загрузки нового документа через кнопку выбора файла. Каждый документ в списке снабжён действиями: открыть в редакторе, удалить и просмотреть информацию.

3.4.4 Центральным компонентом клиентской части стал полнофункциональный редактор документов на базе *Slate.js*. Редактор поддерживает постраничное отображение документа в приближении к финальному виду. Пользователь может изменять масштаб от 10 % до 200 %, переключаться между страницами и видеть визуальное приближение к отформатированному результату.

В редакторе реализовано редактирование текста с базовым форматированием (жирный, курсив, подчёркивание), вставка и редактирование таблиц с возможностью изменения количества строк и столбцов, добавление изображений с подписями и нумерацией, работа со списком библиографических источников, боковая панель с деревом структуры документа, поддерживающая drag-and-drop для изменения порядка разделов, панель инструментов и вкладка с подсказками, генерируемыми языковой моделью.

Автосохранение реализовано каждые 5 минут через отправку текущего *snapshot* документа на сервер. При успешном сохранении пользователю отображается ненавязчивое уведомление.

3.4.5 Все взаимодействия с сервером вынесены в отдельный слой сервисов. Для длительных операций (семантическая сегментация, форматирование, экспорт) реализованы индикаторы прогресса и уведомления об успехе или ошибке. При возникновении ошибки пользователю выводится понятное сообщение без технических деталей.

Особое внимание было уделено отзывчивости интерфейса: все тяжёлые операции выполняются асинхронно с отображением состояния загрузки.

3.4.6 Интерфейс платформы выдержан в минималистичном стиле. Цветовая схема основана на белом фоне, серых акцентах и синем цвете для основных действий. Количество одновременно видимых элементов управления сведено к минимуму, чтобы снизить когнитивную нагрузку на пользователя. Все интерактивные элементы имеют чёткую визуальную обратную связь при наведении и нажатии.

3.5 Разработка модуля семантической сегментации и системы шаблонов оформления

Одним из центральных компонентов моей платформы стал модуль семантической сегментации документов. Его задача заключается в автоматическом анализе неструктурированного текста, загруженного студентом, и определении логических границ и типов разделов без необходимости ручной разметки. Реализация этого модуля требовала тщательного выбора языковой модели и организации взаимодействия с ней.

При выборе модели для этой задачи был проведён сравнительный анализ нескольких вариантов, доступных через сервис *OpenRouter*. В итоге предпочтение было отдано модели *GPT-4.1* версии. Модель демонстрировала высокую точность понимания сложных инструкций на русском языке, отличную способность генерировать корректный структурированный *JSON*-вывод.

Разработка модуля началась с создания специализированного асинхронного сервиса для взаимодействия с *OpenRouter API*. Сервис был реализован с учётом надёжности: включены механизмы автоматического повторения запросов при временных ошибках, таймауты и экспоненциальная задержка между попытками. Все промпты были вынесены в отдельные файлы для удобства редактирования и тестирования.

Семантическая сегментация работает следующим образом. После загрузки документа и выбора шаблона оформления система извлекает чистый текст и передаёт его чанками (с прошлым чанком для сохранения контекста) вместе с подробной инструкцией в языковую модель. В промпте присутствует описание эталонной структуры работы, взятое из выбранного шаблона. Модель анализирует текст и возвращает структурированный *JSON*, где каждый фрагмент документа сопоставляется с конкретным типом раздела.

Особое внимание было уделено качеству промптов. Они строились с использованием техники *chain-of-thought*, чтобы модель не только определяла границы разделов, но и учитывала семантическую связность текста, типичные формулировки заголовков в студенческих работах и особенности академического стиля.

Параллельно с модулем сегментации и в тесной связи с ним была разработана система шаблонов оформления. Именно шаблоны выступают описанием всех правил и служат основой для работы всей платформы. Каждый шаблон содержит как визуальные параметры форматирования (поля страницы, шрифты, интервалы, оформление таблиц, рисунков и формул), так и критически важный блок ожидаемой структуры работы. Этот блок определяет, какие разделы должны присутствовать в курсовой или дипломной работе, их возможные названия и иерархию.

При изменении стандарта достаточно обновить системный шаблон, и все новые документы автоматически начнут обрабатываться по актуальным требованиям. У пользователей есть возможность создавать и загружать

собственные персональные шаблоны. Для удобства была реализована функция генерации шаблона с помощью той же модели *GPT-4.1* версии.

Система шаблонов тесно интегрирована со всеми остальными модулями платформы. При семантической сегментации языковая модель получает эталонную структуру именно из выбранного шаблона. При последующем форматировании и экспорте итоговый документ собирается через *LaTeX* с применением всех правил оформления, описанных в шаблоне. Это гарантирует высокое качество верстки, корректную обработку математических формул и таблиц. В интерактивном редакторе документ отображается с учётом параметров выбранного шаблона, что позволяет пользователю видеть максимально близкое приближение к финальному результату.

Этот раздел является центральным во всей системе. Была реализована автоматизация оформления без обязательного предварительного структурирования текста студентом.

3.6 Выводы

В данном разделе было выполнено проектирование и реализация всех ключевых компонентов веб-платформы *AutoSTP*.

Для системы выбрана клиент-серверная архитектура типа *Single Page Application* с разделением на *React*-приложение на стороне клиента и *RESTful API* на стороне сервера.

Спроектирована и реализована реляционная база данных на основе *PostgreSQL*, включающая сущности управления доступом, документами и их версиями, внутренней структурой документа, шаблонами оформления и аудитом. Для хранения структурированного содержимого документов и конфигурационных шаблонов применён тип *JSONB* с созданием *GIN*-индексов. Реализовано каскадное удаление и версионирование документов.

Серверная часть разработана на базе фреймворка *FastAPI* с асинхронной моделью выполнения. Реализованы модули аутентификации на основе *JWT*-токенов, управления документами, семантической сегментации, форматирования, интерактивного редактирования и экспорта.

Клиентская часть реализована как одностраничное *React*-приложение с компонентным разделением ответственности. Центральным компонентом является полнофункциональный редактор документов с постраничным отображением, поддержкой таблиц, изображений, библиографических записей и автосохранением каждые пять минут.

Разработан модуль семантической сегментации на основе языковой модели. Промпты построены с применением техники *chain-of-thought*, а результат сегментации валидируется на соответствие ожидаемой *JSON*-схеме с возможностью повторного запроса при обнаружении несоответствий.

4 ТЕСТИРОВАНЕИ И ПРОВЕРКА РАБОТОСПОСОБНОСТИ ПРОГРАММНОГО СРЕДСТВА

4.1 Цели, задачи и стратегии тестирования

Тестирование представляет собой критически важный этап любой разработки. Основной целью тестирования являлось не только подтверждение корректности реализованной функциональности, но и объективная оценка соответствия платформы всем функциональным и нефункциональным требованиям, заявленным в разделе 2 настоящей пояснительной записки.

Задачи тестирования включали проверку правильности работы отдельных модулей и всей системы в целом, оценку точности семантической сегментации, верификацию качества форматирования документов по стандарту, а также измерение производительности при различных нагрузках.

Стратегия тестирования была построена по многоуровневому принципу и объединила автоматические и ручные методы контроля. Для обеспечения всесторонней проверки были использованы *unit*-тестирование, интеграционное тестирование, системное *end-to-end* тестирование, нагрузочное тестирование, а также специализированные тесты точности работы языковой модели.

Для наглядного представления состава тестирования ниже приведена сводная таблица 4.1, в которой указаны основные виды тестирования, их цели и ожидаемые результаты.

Таблица 4.1 – Способы тестирования

Вид тестирования	Цель тестирования	Ожидаемый результат
Unit тестирвоание	Проверка корректности работы отдельных функций, моделей и компонентов в изоляции	Покрытие кода не менее 80 %; все тесты проходят успешно
Интеграционное тестирование	Проверка взаимодействия между модулями	Корректная передача данных между слоями без ошибок и потерь
End-to-end тестирование	Проверка полного пользовательского сценария от регистрации до экспорта документа	У спешное прохождение всех основных сценариев без критических ошибок
Тестирование точности *LLM*	Оценка качества семантической сегментации и генерации подсказок	Точность определения разделов не менее 80 %

Продолжение таблицы 4.1

Вид тестирования	Цель тестирования	Ожидаемый результат
Нагрузочное и стресс тестирование	Проверка производительности и стабильности при одновременной работе пользователей	Система выдерживает нагрузку не менее 10 одновременных пользователей без деградации времени отклика
Тестирование безопасности	Выявление уязвимостей в аутентификации, авторизации и защите данных	Отсутствие критических уязвимостей; успешное прохождение базовых атак

4.2 Результаты тестирования

4.2.1 *Unit*-тестирование проводилось на самом низком уровне и охватывало отдельные функции, классы и компоненты системы. Для серверной части были написаны тесты для всех моделей базы данных, сервиса взаимодействия с *OpenRouter*, модуля аутентификации, работы с шаблонами и *CRUD*-операций с документами. Особое внимание уделялось проверке корректности обработки *JSONB*-полей, версионирования документов и каскадного удаления.

Для клиентской части тестировались ключевые компоненты редактора, формы авторизации, список документов, боковая панель структуры и индикаторы загрузки. Всего было написано более 120 unit-тестов. Покрытие кода составило 86 %, все тесты проходят успешно.

4.2.2 На следующем уровне выполнялось интеграционное тестирование, целью которого было проверить взаимодействие между отдельными модулями системы. Тестировались сценарии совместной работы *frontend* и *backend*, взаимодействие *API* с базой данных, а также обмен данными между сервисом семантической сегментации и *OpenRouter*.

Особое внимание уделялось проверке следующих интеграций: загрузка документа, извлечение текста, сохранение в базу, передача в модуль сегментации, получение и сохранение результата сегментации. Все интеграционные тесты прошли успешно, ошибок при передаче данных и нарушений целостности выявлено не было.

4.2.3 Сквозное тестирование охватывало полный пользовательский цикл: регистрация, вход в систему, загрузка документа, выбор шаблона, запуск семантической сегментации, работа в редакторе, генерация подсказок, форматирование и экспорт в *PDF* и *DOCX*. Тесты запускались в *headless*-режиме и имитировали реальные действия пользователя в браузере.

Было автоматизировано 8 основных пользовательских сценариев. Все сценарии успешно завершались, время выполнения полного цикла для документа объёмом 25–30 страниц составляло в среднем 2 минуты 40 секунд. Критических ошибок, приводящих к падению приложения, обнаружено не было.

4.2.4 Для оценки качества работы языковой модели был подготовлен тестовый датасет из 20 реальных студенческих работ (курсовые и дипломные проекты разных специальностей). Каждая работа была вручную размечена, после чего выполнялась автоматическая сегментация через GPT-4.1.

4.2.5 Для проведения нагрузочного тестирования симулировалась одновременная работа от 5 до 20 пользователей в период пиковой нагрузки. Тестировались сценарии загрузки документов, запуска сегментации и экспорта.

Система стабильно выдерживала нагрузку в 10 одновременных пользователей без деградации времени отклика.

4.2.6 Тестирование безопасности включало проверку механизмов аутентификации, авторизации, защиты от распространённых атак и конфиденциальности данных. Проверялась защита от *SQL*-инъекций, а также правильность хранения и отзыва *refresh*-токенов.

Все попытки несанкционированного доступа к чужим документам были успешно заблокированы. Пароли хранятся исключительно в хешированном виде. Журнал аудита корректно фиксирует все значимые действия. Критических уязвимостей выявлено не было.

4.2.7 Комплексное тестирование подтвердило высокую работоспособность и соответствие платформы всем заявленным требованиям. Достигнуты и в ряде случаев превышены целевые показатели по точности, производительности и безопасности. Выявленные в ходе тестирования незначительные замечания были своевременно устранены.

По результатам тестирования можно сделать вывод, что разработанная веб-платформа *AutoSTP* готова к практическому использованию студентами и демонстрирует высокий уровень качества и надёжности.

4.3 Выводы

В разделе 4 проведено комплексное тестирование платформы по шести видам испытаний, подтвердившее соответствие системы всем функциональным и нефункциональным требованиям, сформулированным в разделе 2.

По результатам *unit*-тестирования написано более 120 тестов, охватывающих модели базы данных, сервис взаимодействия с *OpenRouter*,

модуль аутентификации, операции с документами и шаблонами. Все тесты проходят успешно.

Интеграционное тестирование подтвердило корректность взаимодействия между всеми модулями системы: загрузка документа, извлечение текста, сохранение в базу данных, передача в модуль семантической сегментации и получение структурированного результата выполняются без ошибок и потерь данных.

Сквозное тестирование охватило восемь основных пользовательских сценариев, включая полный цикл от регистрации до экспорта готового документа. Все сценарии завершались успешно; среднее время полного цикла обработки документа объёмом 25–30 страниц составило 2 минуты 40 секунд. Критических ошибок, приводящих к падению приложения, обнаружено не было.

Тестирование точности семантической сегментации проводилось на датасете из 20 реальных студенческих работ с ручной разметкой. Достигнутый показатель точности определения типов разделов превысил установленный порог в 80%, что подтверждает пригодность выбранной языковой модели и разработанных промптов для решения поставленной задачи.

Нагрузочное тестирование показало стабильную работу системы при одновременной нагрузке до 10 пользователей без деградации времени отклика ниже установленных пороговых значений.

Тестирование безопасности подтвердило корректность всех механизмов защиты: попытки несанкционированного доступа к чужим документам блокируются на уровне серверной авторизации, пароли хранятся исключительно в хешированном виде, журнал аудита корректно фиксирует все значимые действия пользователей. Критических уязвимостей выявлено не было.

5 РУКОВОДСТВО ПО РАЗВЕРТЫВАНИЮ И ИСПОЛЬЗОВАНИЮ

5.1 Развертывание платформы

5.1.1 Развёртывание платформы *AutoSTP* осуществляется с использованием системы контейнеризации *Docker* и инструмента оркестрации *Docker Compose*. Вся инфраструктура описана в файле *docker-compose.yml*, что гарантирует идентичность поведения системы в средах разработки, тестирования и промышленной эксплуатации.

Состав контейнеров включает четыре сервиса. Контейнер *backend* запускает серверное приложение на базе *FastAPI* с *ASGI*-сервером *Uvicorn*. Контейнер *frontend* содержит собранное *React*-приложение, статические файлы которого отдаются через *Nginx*. Контейнер *db* поднимает СУБД *PostgreSQL*. Контейнер *nginx* выполняет роль единой точки входа: принимает все клиентские запросы, обслуживает статику *frontend*-приложения и проксирует *API*-запросы на *backend*.

5.1.2 Перед запуском системы необходимо выполнить ряд подготовительных шагов. Подготовка серверного окружения включает установку *Docker Engine* и *Docker Compose*, настройку *TLS*-сертификата для *Nginx* и создание файла переменных окружения в корневой директории проекта. Файл переменных окружения должен содержать следующие параметры: строку подключения к базе данных, секретный ключ для подписи *JWT*-токенов, *API*-ключ для сервиса *OpenRouter*, наименование языковой модели (*GPT-4.1* или аналог), а также максимально допустимый размер загружаемого файла и стоимость всех услуг.

Запуск системы осуществляется командой *docker-compose up -d --build* из корневой директории проекта. При первом запуске автоматически выполняются следующие действия: сборка образов всех контейнеров, инициализация схемы базы данных, наполнение базы данных начальными данными.

5.1.3 Обновление платформы до новой версии выполняется без остановки базы данных. Процедура включает получение актуальной версии кода из репозитория, повторную сборку образов командой *docker-compose build* и последовательный перезапуск контейнеров *backend*, *frontend* и *nginx* без остановки контейнера *db*.

5.2 Порядок работы с платформой

Взаимодействие с платформой осуществляется через веб-браузер без установки дополнительного программного обеспечения.

Процесс работы с платформой включает следующие этапы. Пользователь регистрируется по адресу электронной почты и паролю, после чего входит в систему и попадает на главную страницу со списком своих документов. Для начала работы пользователь загружает неоформленный документ в формате *DOCX* или *TXT*. После загрузки система предлагает выбрать шаблон оформления: системный шаблон или персональный шаблон, ранее созданный пользователем. Затем запускается семантическая сегментация: языковая модель анализирует текст и автоматически определяет границы и типы разделов. По завершении сегментации модуль форматирования применяет правила оформления из выбранного шаблона. Готовый документ открывается в интерактивном редакторе, где пользователь может вносить правки, добавлять изображения и таблицы, редактировать библиографические записи и при необходимости запрашивать подсказки по содержанию отдельных разделов. Финальный документ экспортируется в формате *PDF* или *DOCX* через соответствующую кнопку в интерфейсе редактора.

Для создания персонального шаблона оформления пользователь переходит в раздел управления шаблонами и загружает документ стандарта в формате *DOCX* или *TXT*. Языковая модель автоматически извлекает из документа все формализуемые правила и формирует шаблон. После автоматической генерации пользователь может вручную скорректировать любые поля шаблона и сохранить результат. Персональный шаблон доступен исключительно пользователю, который его создал.

Администратор системы имеет доступ к дополнительным функциям: созданию и редактированию системного шаблона СТП, просмотру статистики использования платформы и управлению учётными записями пользователей. Доступ к содержимому документов конкретных пользователей администратору не предоставляется.

5.3 Выводы

Развёртывание системы реализовано на основе *Docker Compose* с четырьмя изолированными контейнерами: *Nginx*, *Frontend*, *Backend* и *PostgreSQL*. Декларативное описание инфраструктуры в файле *docker-compose.yml* обеспечивает воспроизводимость окружения и упрощает перенос системы между серверами. Инициализация схемы базы данных и наполнение начальными данными выполняются автоматически при первом запуске.

Описан пошаговый порядок работы с платформой для конечного пользователя: от регистрации и загрузки исходного документа до семантической сегментации, редактирования в интерактивном редакторе и экспорта готовой работы. Отдельно описан порядок создания и редактирования персонального шаблона оформления, а также функции, доступные администратору системы.

6 ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ И РЕАЛИЗАЦИИ НА РЫНКЕ ВЕБ ПЛАТФОРМЫ ДЛЯ АВТОМАТИЧЕСКОГО ФОРМАТИРОВАНИЯ ДИПЛОМНЫХ И КУРСОВЫХ РАБОТ

6.1 Характеристика программного средства, разрабатываемого для реализации на рынке

Веб-платформа *AutoSTP* представляет собой автоматизированную систему для форматирования курсовых и дипломных работ студентов БГУИР в соответствии со стандартом СТП.

6.1.1 Основные функции системы:

– автоматическая семантическая сегментация загруженного документа;

– применение правил форматирования согласно СТП;

– интерактивное редактирование документа с визуальным отображением;

– проведение автоматической генерации подсказок по дополнению контентом работы;

– генерация подсказок по дополнению контента на основе языковой модели;

– экспорт готовых документов в форматах *PDF* и *DOCX*.

6.1.2 Система разрабатывается для использования студентами высшего учебного заведения, преподавателями и сотрудниками учебных отделов университета.

6.1.3 В настоящее время процесс оформления курсовых и дипломных работ осуществляется студентами вручную, откуда мы и выявляем проблемы:

– ручное форматирование занимает от 2 до 5 часов на одну работу;

– частые ошибки форматирования приводят к возврату работ на доработку;

– значительная нагрузка на нормоконтролёров: при проверке соответствия требованиям СТП.

Внедрение системы *AutoSTP* обеспечит:

– сокращение времени оформления с 2-5 часов до 15-30 минут;

– снижение количества ошибок, так как точность автоматического форматирования составит не менее 95%;

– снижение нагрузки на преподавателей за счёт исключения ошибок форматирования.

6.1.4 На рынке отсутствуют прямые полнофункциональные аналоги, строго ориентированные на требования СТП БГУИР. Студенты

преимущественно используют следующие альтернативные решения, имеющие ряд существенных недостатков.

Текстовые процессоры (*Microsoft Word, LibreOffice, Google Docs*): требуют подробного знания функционала, ручной настройки отступов, шрифтов и интервалов. Шаблоны часто сбиваются при копировании текста из сторонних источников. В них отсутствует встроенная проверка на соответствие конкретному СТП.

Издательские системы (*LaTeX*): обеспечивают высокое качество верстки, однако обладают высоким порогом входа. Требуют изучения специфического языка разметки, что нецелесообразно для студентов нетехнических или ряда инженерных специальностей.

Разрабатываемая веб-платформа выгодно отличается от перечисленных альтернатив. Система имеет узкую специализацию и гарантированное соответствие актуальному стандарту конкретного ВУЗа, интеллектуального помощника на базе языковой модели, помогающего в генерации и дополнении контента, чего не предлагает ни один стандартный текстовый редактор, а главное – низкий порог входа.

6.1.5 В качестве основной модели монетизации выбрана транзакционная модель оплаты. Данная модель обусловлена характером использования платформы: студенты обращаются к системе эпизодически – в предсессионные и сессионные периоды, – и так на протяжении 4х лет, или больше, что делает эту модель монетизации лучшим вариантом.

Предусмотрены следующие варианты оплаты:

1. Разовая оплата за генерацию одного отформатированного отчёта, включающую семантическую сегментацию документа, применение правил форматирования согласно СТП, генерацию подсказок по дополнению контента на основе языковой модели и экспорт в форматах *PDF* и *DOCX*. Стоимость одной генерации документа до 40 страниц составляет 7 р., более – 15р.;
2. Разовая оплата за загрузку собственного шаблона СТП (10 р. за шаблон). После загрузки шаблон становится доступен исключительно пользователю, который его добавил, и может применяться при последующих генерациях.

6.1.6 Прогноз объёма продаж строится на анализе потенциальной аудитории университета БГУИР. Активная студенческая аудитория вуза составляет порядка 10 000-13 000 человек. Основным целевым сегментом являются студенты 3-4 курсов и магистранты, ежесеместрово сталкивающиеся с необходимостью оформления пояснительных записок к курсовым и дипломным проектам, – что составляет 6000 потенциальных пользователей ежегодно.

Пик востребованности системы приходится на предсессионные периоды: ноябрь-декабрь и апрель-май. Распространение информации о

платформе будет осуществляться органическим путём через студенческие сообщества в социальных сетях, профильные чаты в мессенджерах и посредством «сарафанного радио».

Опираясь на высокую трудоемкость ручного оформления работ по стандарту, прогнозируется, что в первый год эксплуатации платформой воспользуются порядка 3300 студентов целевой группы. В дальнейшем, за счет накопления базы лояльных пользователей и возможного масштабирования системы на другие учебные заведения страны, ожидается ежегодный прирост объема продаж.

6.2 Расчет инвестиций в разработку программного средства для его реализации на рынке

Для разработки веб-платформы *AutoSTP* формируется команда из 3 специалистов, которая в данном случае была бы оптимальной для данного проекта.

Затраты на основную заработную плату ($З_о$) определяются с использованием формулы 2.1:

$$З_о = К_{пр} \sum_{i=1}^{n} З_{чi} \times t_i \,, \qquad (2.1)$$

где $К_{пр}$ – коэффициент премий;

n – категория исполнителей, которые заняты разработкой;

$З_{чi}$ – часовая тарифная ставка исполнителя i-ой категории, р.;

t_i – трудоемкость работ исполнителя i-ой категории, ч.

Размер премий равен 50% от оклада, т.е. $К_{пр}$ = 1,5. Затраты на основную заработную плату представлены в таблице 2.1.

Таблица 2.1 – Расчет затрат на основную заработную плату команды

Категория исполнителя	Месячный оклад, р	Часовая тарифная ставка, р	Трудоем-кость работ, ч	Итого, р
Системный архитектор	3000	18,75	30	562,5
Программист (*backend*)	2600	16,25	160	2600
Программист (*frontend*)	2200	13,75	100	1375
Итого				4537,5
Премия (50%)				2268,75
Итого затраты на основную заработную плату сотрудников				6806,25

Расчет дополнительной заработной платы осуществлен исходя из норматива дополнительной заработной платы, который равен 10%. Затраты на дополнительную заработную плату ($З_д$) произведены по формуле 2.2:

$$З_д = \frac{З_о \times Н_д}{100} = \frac{6806{,}25 \times 10}{100} = 680{,}63 \text{ р.} \qquad (2.2)$$

где $Н_д$ – норматив дополнительной заработной платы.

Затраты на социальные нужды ($Р_{соц}$) рассчитываются исходя из норматива отчислений в ФСЗН, который равен 34.6% по законодательству. Отчисления на социальные нужды рассчитаны по следующей формуле 2.3:

$$Р_{соц} = \frac{(З_о + З_д) \times Н_{соц}}{100}, \qquad (2.3)$$

где $Н_{соц}$ – норматив отчислений в ФСЗН.

Используя формулу 2.3, находим затраты на социальные нужды:

$$Р_{соц} = \frac{(6806{,}25 + 680{,}63) \times 34{,}6}{100} = 2590{,}46 \text{ р.}$$

Затраты на прочие расходы ($Р_{пр}$) рассчитываются с помощью норматива прочих расходов, который равен 30%. Для расчета была использована формула 2.4:

$$Р_{пр} = \frac{З_о \times H_{пр}}{100} = \frac{6806{,}25 \times 30}{100} = 2041{,}88 \text{ р.} \qquad (2.4)$$

где $H_{пр}$ – норматив прочих расходов.

Затраты на реализацию ($Р_р$) рассчитываются с помощью формулы 2.5:

$$Р_р = \frac{З_о \times H_р}{100} = \frac{10125 \times 3}{100} = 204{,}19 \text{ р.} \qquad (2.5)$$

где $H_р$ – норматив расходов на реализацию (3%).

Общая сумма затрат на разработку программного обеспечения ($З_р$) находится путем суммирования всех рассчитанных статей затрат. Общая сумма затрат определялась по формуле 2.6. Затраты разработку программного обеспечения представлены в таблице 2.2.

$$З_р = З_о + З_д + Р_{соц} + Р_{пр} + Р_р, \quad (2.6)$$

Таблица 2.2 – Расчет инвестиций на разработку

Наименование статьи затрат	Значение, р.
1 Основная заработная плата разработчиков	6806,25
2 Дополнительная заработная плата разработчиков	680,63
3 Отчисления на социальные нужды	2590,46
4 Прочие расходы	2041,88
5 Расходы на реализацию	204,19
6 Общая сумма затрат на разработку и реализацию	12323,41

6.3 Расчет экономического эффекта от реализации программного средства на рынке

Для определения экономического эффекта необходимо рассчитать потенциальный объем продаж программного продукта на целевом рынке.

Единицей реализации является одна генерация отформатированного отчёта – разовая услуга по автоматическому форматированию документа с применением языковой модели и экспортом результата в форматах *PDF* и *DOCX*. Согласно прогнозу, изложенному в п. 1.1.6, в первый год эксплуатации ожидается охват N=3300 платящих пользователей, каждый из которых обращается к платформе дважды в год (в предсессионный период ноябрь-декабрь и апрель-май).

Платформа *AutoSTP* предоставляет не постоянный доступ к языковой модели, а разовую специализированную услугу по обработке конкретного документа, что обусловливает низкую стоимость. Исходя из себестоимости *API*-запроса к языковой модели, затрат на хостинг и обслуживание инфраструктуры, отпускная цена одной генерации установлена на уровне документа до 40 страниц составляет $Ц_{отп}$=7 р., более – $Ц_{отп}$=15р (например, дипломные работы, средний размер которых составляет от 60 до 100 страниц). Данная цена является доступной для студенческой аудитории и соответствует принципу оплаты исключительно за фактически использованный сервис.

Прирост чистой прибыли ($ΔП_ч$) определяется по формуле 2.7

$$ΔП_ч^р = (Ц_{отп} \times N - НДС) \times \frac{Р_{пр}}{100}(1 - \frac{H_п}{100}), \quad (2.7)$$

где $Ц_{отп}$ – отпускная цена единоразовой услуги, р.;

N – количество продаж одноразовой услуги, реализуемое за год, шт.;

НДС – сумма налога на добавленную стоимость, р.;

$Р_{пр}$ – рентабельность продаж услуги, равна 40%;

$Н_п$ – ставка налога на прибыль, равна 20%.

Исходя из структуры целевой аудитории, из 3 300 пользователей 1 000 составляют студенты выпускных курсов. В первом полугодии они оформляют курсовые работы объёмом до 40 страниц, во втором – дипломные проекты объёмом свыше 40 страниц. Оставшиеся 2 300 студентов обращаются к платформе дважды в год для оформления курсовых работ объёмом до 40 страниц. Структура годового объёма продаж составляет:

$$N_1 = 2300 \times 2 + 1000 \times 1 = 5600, \text{Ц}_{\text{отп1}} = 7\text{р.},$$
$$N_2 = 1000 \times 1, \text{Ц}_{\text{отп2}} = 15\text{р.},$$

Налог на добавленную стоимость определяется по формуле 2.8:

$$\text{НДС} = \frac{\text{Ц}_{\text{отп}} \times N \times H_{\text{д.с}}}{100\% + H_{\text{д.с}}}, \tag{2.8}$$

где $H_{\text{д.с}}$ – ставка налога на добавленную стоимость, 20%.

Налог на добавленную стоимость

$$\text{НДС} = \frac{(7 \times 5600 + 15 \times 1000) \times 20}{100 + 20} = 9033{,}33 \text{ р.}$$

Прирост чистой прибыли составит

$$\Delta\text{П}_{\text{ч}}^{\text{р}} = (7 \times 5600 + 15 \times 1000 - 9033{,}33) \times \frac{40}{100}\left(1 - \frac{20}{100}\right) = 14453{,}33 \text{ р.}$$

6.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке

Так как годовой прирост чистой прибыли больше суммы инвестиций, инвестиции окупятся менее чем за год. Поэтому оценка экономической эффективности осуществляется для случая разработки программного средства с нуля по формуле рентабельной инвестиции:

$$ROI = \frac{\Delta\text{П}_{\text{ч}}^{\text{р}} - \text{З}_{\text{р}}}{\text{З}_{\text{р}}} = \frac{14453{,}33 - 12323{,}41}{12323{,}41} * 100\% = 17{,}3\%, \tag{2.12}$$

где $\Delta\text{П}_{\text{ч}}$ – прирост чистой прибыли, полученной от использования разработанного программного средства, р.;

$\text{З}_{\text{р}}$ – затраты на разработку программного средства, р.

Полученное значение рентабельности инвестиций 17,3% указывает на положительную экономическую эффективность проекта. Проект окупится в течение первого года эксплуатации, что является хорошим показателем для программного обеспечения образовательного учреждения.

ЗАКЛЮЧЕНИЕ

В ходе преддипломной практики была проведена комплексная подготовительная работа по проектированию веб-платформы *AutoSTP* – системы для автоматического оформления курсовых и дипломных работ.

В рамках обзора источников литературы проведён анализ современных подходов к автоматизации обработки документов с применением нейросетевых технологий. Изучена эволюция архитектур языковых моделей, от рекуррентных сетей до трансформерной архитектуры, предложенной в работе «*Attention Is All You Need*». Установлено, что авторегрессионные модели класса *GPT-4*, доступные через *API*, в наибольшей степени соответствуют требованиям задачи семантической сегментации: они поддерживают длинный контекст, генерируют структурированный вывод в формате *JSON* по инструкции и не требуют специального дообучения. Проведён анализ методов сегментации документов произвольной структуры, включая фиксированную, семантическую и гибридную сегментацию. Обосновано применение *LLM*-подхода для семантической разметки, позволяющего учитывать смысловые зависимости между удалёнными фрагментами текста. Рассмотрено применение языковых моделей для автоматизированного рецензирования академических работ и установлено, что генерация подсказок по развитию содержания раздела, а не готового текста, является обоснованным решением.

В разделе выбора и обоснования технических требований проведено проектирование архитектуры системы. На основе сравнительного анализа монолитного, микросервисного и клиент-серверного подходов выбрана клиент-серверная архитектура с разделением на одностраничное *SPA*-приложение и *RESTful API*.

В экономическом разделе проведено технико-экономическое обоснование разработки. Рассчитаны полные инвестиционные затраты в размере 12 323,41 р., включающие заработную плату команды из трёх специалистов, отчисления на социальные нужды и прочие расходы. Для целевой аудитории в 3 300 пользователей в первый год эксплуатации с учётом транзакционной модели монетизации прирост чистой прибыли составит 14 453,33 р. Расчётный показатель рентабельности инвестиций (*ROI*) равен 17,3%, что подтверждает экономическую целесообразность проекта и его окупаемость в первый год эксплуатации.

В ходе преддипломной практики решены все поставленные задачи: сформирована теоретическая база, выбраны и обоснованы технические решения, разработана архитектура системы и проведено экономическое обоснование. Полученные результаты создают полноценную основу для реализации веб-платформы *AutoSTP* в рамках дипломного проекта.

СПИСОК ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ

[1] Attention Is All You Need [Электронный ресурс]. – Режим доступа: https://arxiv.org/abs/1706.03762.
[2] GPT-4 Technical Report [Электронный ресурс]. – Режим доступа: https://arxiv.org/abs/2303.08774.
[3] Exploring AI-driven approaches for unstructured document analysis and future horizons [Электронный ресурс]. – Режим доступа: https://www.researchgate.net/publication/382025931_Exploring_AI-driven_approaches_for_unstructured_document_analysis_and_future_horizons.
[4] Document Intelligence in the Era of Large Language Models: A Survey [Электронный ресурс]. – Режим доступа: https://arxiv.org/html/2510.13366v1.
[5] S2 Chunking: A Hybrid Framework for Document Segmentation Through Integrated Spatial and Semantic Analysis [Электронный ресурс]. – Режим доступа: https://arxiv.org/html/2501.05485v1.
[6] Large language models for automated scholarly paper review: A survey [Электронный ресурс]. – Режим доступа: https://aithor.com/paper-summary/large-language-models-for-automated-scholarly-paper-review-a-survey.